

Pathfinding Comparator

Report

Computer Games – Phase B, Topic: Path Planning

Group: 1 **Members:** Franziska Plagwitz, Luna Schnellbacher, Sem Massa

Introduction

The Phase B assignment asks for a game demonstrating at least three lecture techniques plus one recently developed technique, documented in one organized report with references. Our project, the *Pathfinding Comparator*, lets three non player characters race or live inside a procedurally generated house while each one is driven by a different search algorithm, so the algorithms can be compared directly instead of only in theory. This report documents four techniques: path planning and navigation (A*, D* Lite, and directed Jump Point Search as the required recently developed technique), game engine architecture (a finite state machine), collision detection (broad phase and narrow phase tests), and sprite based keyframe animation of the non player characters. For each technique this report gives the underlying idea from the lecture and then the exact file, class and function where it is implemented.

1 User Guide

The game is a single Godot 4.6 project (GL Compatibility renderer), `res://scenes/Main.tscn` as the main scene. Opening the project and pressing play is all that is required.

- **Start panel** (`scripts/start_ui.gd`), shown on launch: choose *Race* mode (one timed run to a shared goal) or *Life* mode (non player characters cycle endlessly between three hotspots), an obstacle count, and which of the three algorithms (A*, D* Lite, JPS) should run. Every ticked algorithm gets its own grid, all sharing the same maze, obstacle routes and start/goal cells for a fair comparison.
- **Left click** on a walkable cell places the shared spawn point. In Race mode a second left click sets the goal and starts the run. In Life mode the non player characters immediately start cycling through their hotspots.
- **O key** toggles a per grid overlay (`scripts/grid.gd`, `toggle_overlay()`) showing every explored cell plus the resulting route, the clearest way to see how the algorithms search differently.
- **Right click** restarts the scene with a freshly generated maze.
- **Results:** Race mode shows a dashboard (`scripts/benchmark_dashboard.gd`) per algorithm with time, explored cells, path length and an efficiency ratio (explored divided by path length,

lower is more focused). Life mode shows a live panel (`scripts/life_dashboard.gd`) with hotspots reached and hotspots per minute.

2 Technique 1: Path Planning and Navigation (A*, D* Lite, JPS+)

2.1 Background

The lecture introduces A* as an informed search expanding the node with lowest $f(n) = g(n) + h(n)$ first, $g(n)$ being the cost already paid and $h(n)$ an admissible heuristic estimate of the remaining cost. On an eight directional grid the octile heuristic (orthogonal steps cost one, diagonal steps cost $\sqrt{2}$) is admissible and thus keeps A* optimal. The same lecture covers dynamic path planning for environments that change while an agent moves, and explicitly names D* as a variant of A* extended with dynamic blocking support. D* Lite keeps two cost estimates per cell, g and rhs , and repairs only the part of the plan a changed cell actually affects instead of solving the whole search again.

The exercise sheet additionally requires one recently developed technique. For this project that role is filled by Jump Point Search (JPS), specifically the directed, preprocessing based JPS+ variant from a 2023 IEEE publication, named directly in the project plan as the chosen recent technique. Instead of expanding every neighboring cell one step at a time, JPS prunes neighbors using the local grid shape and jumps ahead in a straight line until reaching a cell that forces a direction change, a jump point, sharply reducing the number of cells kept in the open list.

2.2 Implementation

All three share the interface in `algorithms/pathfinder.gd`, class `Pathfinder`: shared costs `ORTHOGONAL_COST = 1.0` and `DIAGONAL_COST = 1.41421356`, the octile heuristic, and a `BlockReaction` enum (`RETREAT/REPLAN`) that lets a subclass declare how its non player character reacts once blocked.

A*, `algorithms/astar.gd` (class `AStarPathfinder`): `find_path()` (line 20) runs the textbook algorithm with an open set, `g_score` and `came_from`, recording every expanded cell into `explored_cells` for the overlay. `block_reaction()` (line 15) returns `RETREAT`.

D* Lite, `algorithms/dstar_lite.gd` (class `DStarLitePathfinder`): keeps `_g`, `_rhs`, a priority queue `_pqueue` and an offset key `_km` between calls. `find_path()` (line 45) calls `_initialize()` then `_compute_shortest_path()` following the standard key comparison and vertex update rules. `block_reaction()` (line 38) returns `REPLAN`: instead of retreating, it recomputes from the current cell.

JPS+, `algorithms/jps.gd` (class `JPSPathfinder`): `find_path()` (line 37) follows the classic pruning and jumping scheme with one deliberate change: the standard JPS forced neighbor rule assumes diagonal corner cutting is legal, while this project's grid forbids it (`diagonal_move_allowed()` in `pathfinder.gd`, line 60), so `_pruned_directions`, `_jump` and `_should_stop_at` were adapted by hand for that restriction. A jump cache (`_jump_cache`) avoids repeated jump computations, and `_densify()` expands the sparse jump points back into a dense, per cell path so the rest of the game treats every algorithm's output identically. Its doc comment states it was verified against a from scratch Dijkstra reference across roughly 1500 randomized mazes. `block_reaction()` (line 34) returns `RETREAT`, like A*.

3 Technique 2: Game Engine Architecture (Finite State Machine)

3.1 Background

The AI engine lecture frames intelligent behavior as sensing, deciding and acting, and lists finite state machines as the simplest decision making technique. Following the Mealy machine model shown there, an object occupies exactly one state at a time and transitions to another only in reaction to an event, based on the current state and that event. The lecture also shows an FSM extended with a retreat branch, the same pattern used here once a path is blocked.

3.2 Implementation

`scripts/fsm.gd` (class `FiniteStateMachine`) owns one active state and exposes `change_state(state_name, msg)` (line 34), which calls `exit()` on the old and `enter(msg)` on the new state; `_physics_process()` (line 27) forwards `update(delta)` to whichever state is active. Every state extends `NPCState` (`scripts/npc_state.gd`). The six states, in `scripts/states/`:

- **WalkState**: follows the path; once blocked, reads `block_reaction()` and switches to `RetreatState` or `BlockState` (`_block_state_name()`, line 37), the point where path planning and the FSM meet.
- **RetreatState**: steps back along the known path (A*, JPS).
- **BlockState**: asks the pathfinder to recompute (D* Lite), falls back to `WaitState`.
- **WaitState**: one second timer, then back to `BlockState`.
- **DoneState**: entered on arrival in Race mode, emits `arrived`.
- **LifeWaitState**: Life mode pause, one and a half seconds, emits `life_spot_reached`.

Godot's signal system is the messaging layer between states and `scripts/main.gd`, matching the project plan's decoupled messaging idea.

4 Technique 3: Collision Detection (Broad Phase and Narrow Phase)

4.1 Background

The collisions lecture splits detection into a broad phase, cheaply ruling out pairs that clearly cannot touch using bounding volumes such as AABBs, and a narrow phase, an exact test applied only to the remaining candidates, for example a circle against box test via a closest point construction: clamp the circle's center into the box, compare the distance to that point against the radius.

4.2 Implementation

Both phases are implemented by hand in `scripts/collision_system.gd` (class `CollisionSystem`); Godot's physics engine is intentionally not used, since the assignment asks for the phases themselves. **Broad phase**: `aabb_overlap()` (line 10), AABB against AABB. **Narrow phase**: `circle_intersects_box()` and `circle_vs_box_collides()` (lines 17, 26), the closest point



Figure 1: Key frames of the walk cycle sprite sheet, one direction per row (front, back, side), four frames each. `_build_single_animation()` turns each row into one looping animation, and the active FSM state picks the row through `animate()`.

construction described above, matching the project plan's circle proxies for non player characters and AABBs for obstacles. `push_circle_out_of_box()` (line 36) resolves an actual collision with a minimum translation vector.

The non player character (`scripts/npc.gd`) is a `CharacterBody2D` for its `CollisionShape2D`, but never calls `move_and_slide()`: `_check_obstacle_collisions()` (line 195) calls into `CollisionSystem` directly. Moving obstacles (`scripts/obstacle.gd`) use a 25 by 25 unit AABB and patrol a fixed route, updating occupied cells so D* Lite can detect a newly blocked cell.

5 Technique 4: Animation (Keyframing)

5.1 Background

The lecture on animation names keyframing as the basic animation technique: characterizing individual key poses and interpolating between them.

5.2 Implementation

The simulator uses a discrete, sprite based version of this idea rather than continuous interpolation. `scripts/npc.gd` loads a sprite sheet per direction and builds one animation per state from a start and end frame (`_load_sprite_animations()`, line 219, `_build_single_animation()`, line 264), and each state's `enter()` switches to its own walk or idle cycle through the shared `animate()` helper in `npc_state.gd`. This is the "Key Frame Animation" item from the project plan, driven directly by the same finite state machine described in Technique 2.

6 Findings and Insights

Search footprint. The efficiency ratio (explored divided by path length) is consistently smallest for JPS, since it only expands jump points instead of every cell along the long straight corridors the maze generator tends to produce. A* and D* Lite both expand every intermediate cell, and on short, twisty routes through cluttered rooms the gap narrows since jump points sit closer together there.

Path length and quality. All three return equal length paths for a given start and goal, since none relaxes the eight directional, corner cutting free grid model; A* and JPS share the same admissible octile heuristic, and D* Lite converges to the same optimal cost. The overlay confirms this: routes coincide almost exactly, while explored cell sets differ sharply.

Behavior once blocked. This is the clearest practical difference in how the algorithms react. A* and JPS both retreat and wait for a moving obstacle to pass. D* Lite recomputes only the affected part of its plan and visibly reroutes around a still blocking obstacle far sooner, without the pause in `WaitState` that A* and JPS go through. One would expect this to translate into a clearly higher hotspots per minute rate for D* Lite in Life mode. In practice it does not: across repeated Life mode runs D* Lite reaches roughly the same number of hotspots per minute as A*, since the immediate rerouting mostly saves the short wait time, which turns out to be a small fraction of total travel time on this maze size, while the D* Lite bookkeeping itself adds a bit of overhead per step. The qualitative difference in behavior is real and visible on screen, but it does not show up as a clear quantitative advantage in the benchmark under these conditions.

Takeaway. JPS suits a mostly static environment where a small search footprint matters. D* Lite reacts to a changing environment in a qualitatively different, more proactive way than A* or JPS, but on this maze size and obstacle density that difference did not translate into a measurable hotspots per minute advantage, so its practical benefit here is more about smoother, less interrupted motion than raw throughput. A* remains the simple baseline the other two are compared against.